A Databricks-native agent-assist framework for telecom support

Putting Every Agent's Best Call in Every Agent's Hands

OZMO

Project Breakdown

GOAL 1

Create an ETL pipeline for transcription data.

GOAL 2

Identify key topics and sentiment shifts.

GOAL 3

Develop a model or set of rules based on goals 1 & 2.

OZMO

Three people benefit when a support call goes right

THE CUSTOMER	THE AGENT	OZMO
Wants the issue solved. Without being bounced.	Wants to look competent on every call.	Wants every interaction to feel effortless.

Mentored by David Bates, Kyle Krcmaric, and Matthew Seiler (Ozmo)

OZMO

Support transcripts are structured data hiding in prose

1

REPEATING BEATS

Every call is a sequence of small agent actions: verify, pull bill, run diagnostic.

2

SHARED PATTERNS

Successful calls share patterns that failed calls don't. The pattern is the playbook.

3

NO OFF-THE-SHELF TOOL

No single library extracts beats, scores success, and serves them in real time.

The question isn't WHAT the data says. It's HOW you compose a system that listens.

OZMO

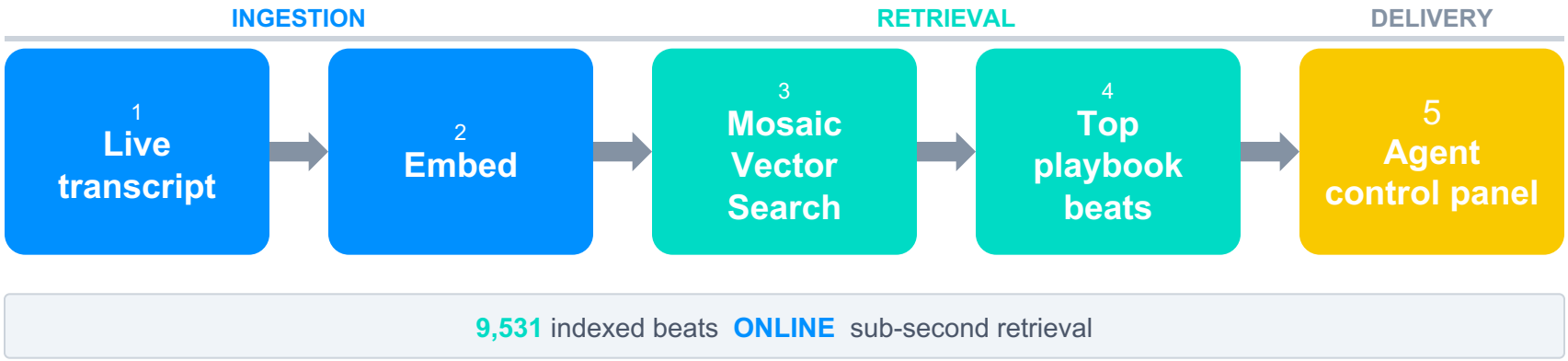
A framework that learns the top 21% of agents' playbook

and offers it to everyone in real time, with the agent in charge.

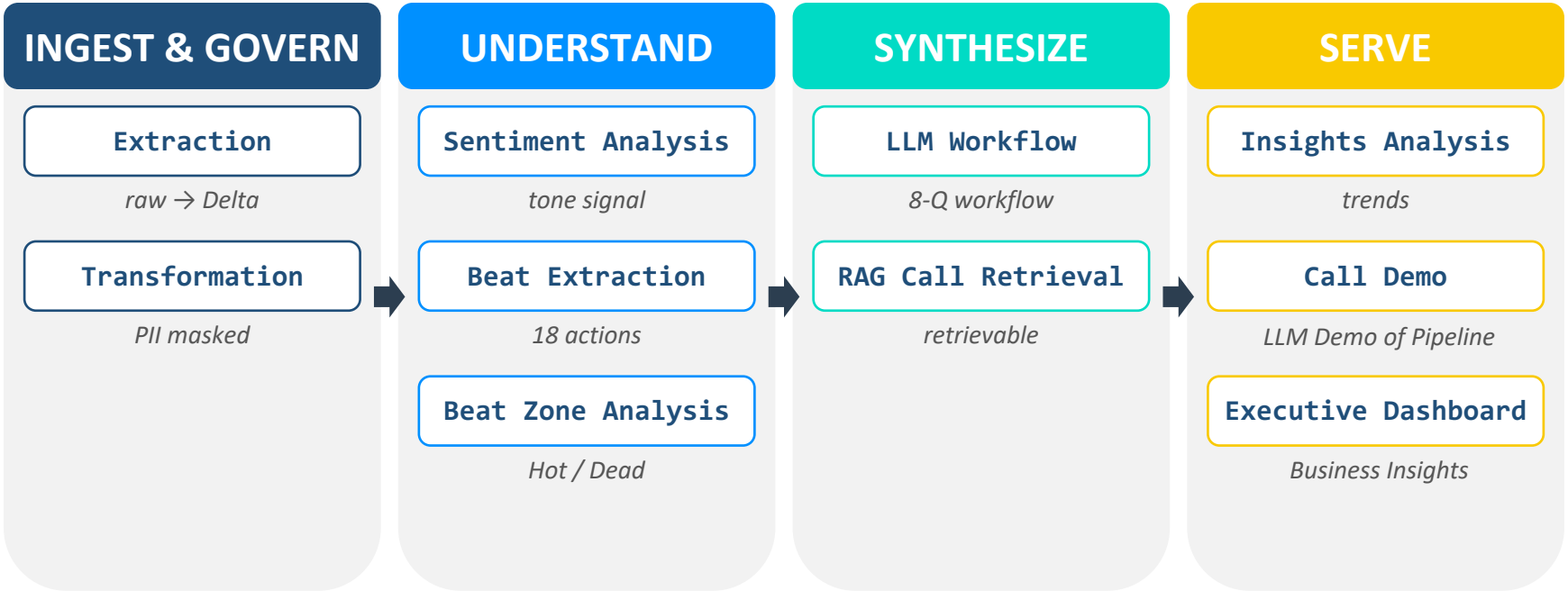
Built end-to-end on Databricks.

OZMO

Mosaic Vector Search delivers the right playbook in under a second



Nine stages, four jobs, one platform

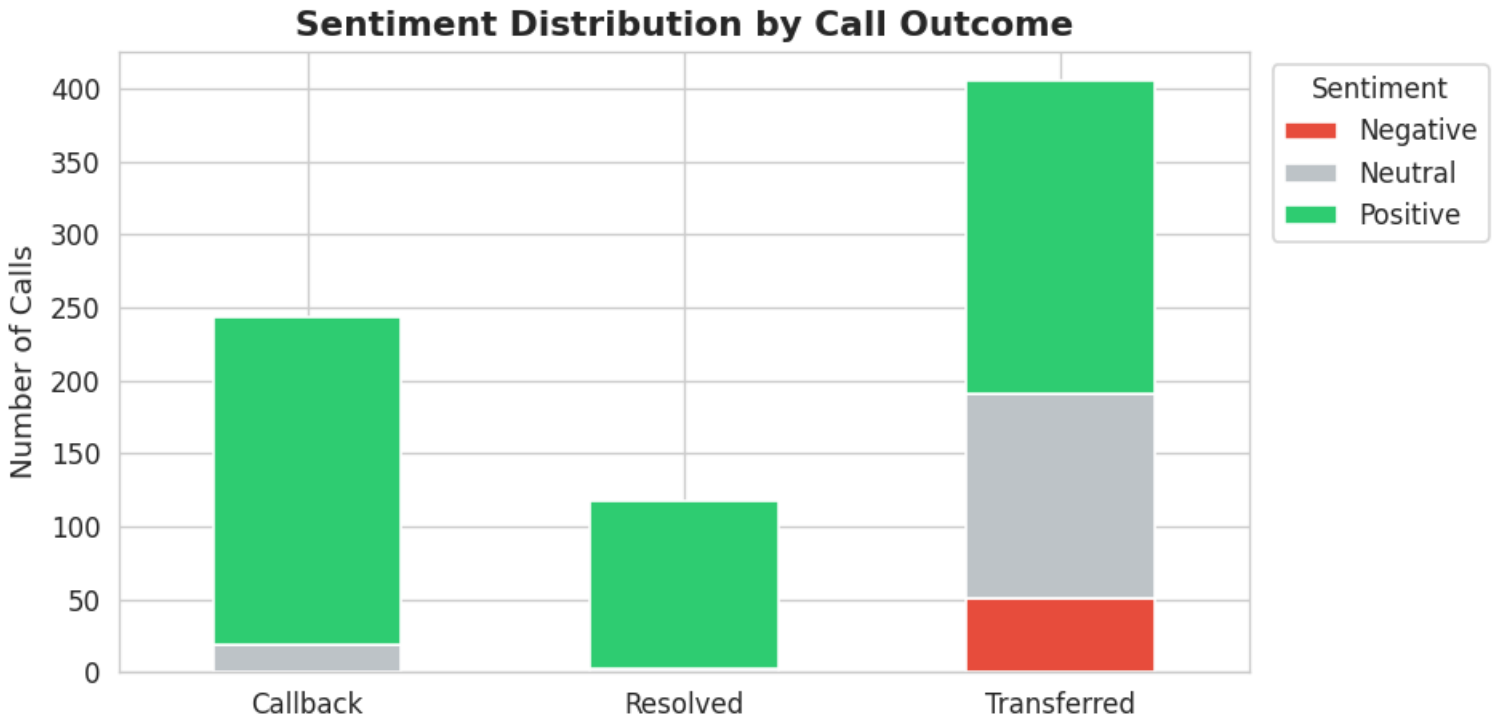


Each stage has one job. Each output is a Delta table the next stage reads.

OZMO

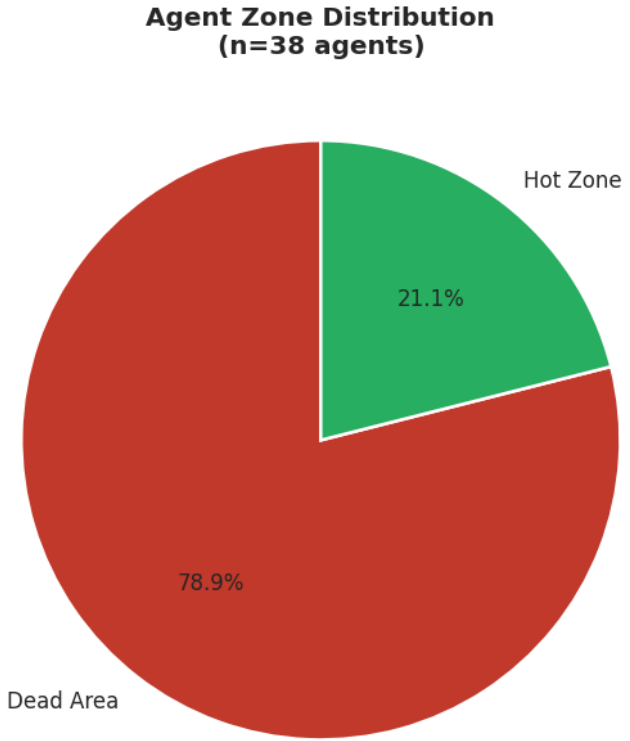
Sentiment Distribution by Call Outcome

Positive sentiment dominates resolved and callback calls; transferred calls show elevated negative and neutral rates



OZMO

21% of agents work from a consistent playbook. 79% improvise.



8

AGENTS

resolve **100%** of their calls

OZMO

Why we built it Databricks-native (and the tradeoff we accepted)

Capability	What we used	Why this choice
PII masking	ai_mask() (SQL)	Runs before any LLM call
Sentiment	ai_analyze_sentiment()	Deterministic, batchable
Storage	Delta Lake	Versioned · time-travelable

TRADEOFF: tight platform coupling

EARNED: zero external infrastructure

OZMO

NOW WE GO TO DASHBOARD

Every call decomposes into 18 reusable agent actions

VERIFY_ACCOUNT_IDENTITY	PULL_BILLING_STATEMENTS	CHECK_DEVICE_MODEL
RUN_REMOTE_DIAGNOSTIC	ENABLE_WIFI_CALLING	CONFIRM_CALL_QUALITY
TRANSFER_TO_RETENTION	ESCALATE_ISSUE	CONFIRM_RESOLUTION

9,604 agent utterances clustered into 18 reusable beats.

Each beat = one action. Calls = sequences of beats.

OZMO

Why QA-CoT: bottom-up meets top-down

BOTTOM-UP

Beats clustered from real agent utterances

What agents ACTUALLY do. Grounded in the corpus.



TOP-DOWN

8 grounded questions per category

What a USABLE workflow looks like.

WHY BOTH HALVES

Combined: a workflow an agent can follow AND that reflects reality.

Humans in the loop: by design, not by accident

Three deliberate decisions make the human position load-bearing:

Decision	Why it's a method choice, not just a guardrail
PII masked BEFORE any model call	Privacy at the data layer · not bolted on later
Three suggestions per call, never one	Forces choice · resists automation drift
Override is logged and reviewed	Wrong suggestions become training signal



OZMO

What surprised us: three findings worth carrying forward

1

TONE ≠ OUTCOME

Sentiment doesn't predict resolution.

Polite calls transfer; frustrated calls resolve.

2

CONSISTENCY > VOLUME

Consistency predicts health better than beat count.

A 5-step playbook beats a 12-step one.

3

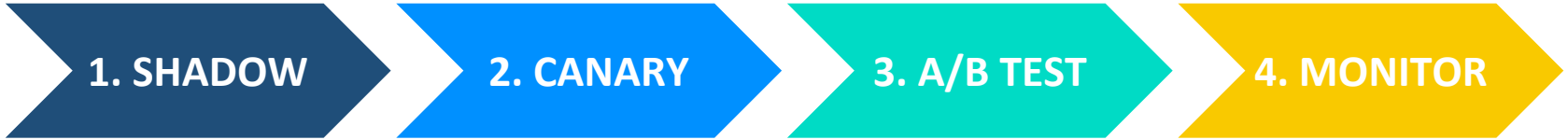
SOFT FALLBACKS > HARD RULES

Strict classifiers starve small categories.

Progressive fallback beats brittle thresholds.

OZMO

Taking This From Capstone to Production



Run alongside live calls *log only*	*1–2 senior agents* *1 category*	*Randomized arms* *AHT · transfer · callback*	*Drift on cluster purity* *retrain triggers*

Re-train on real (non-synthetic) transcripts before canary.

The framework is portable to any transcript-driven vertical (see appendix A14).

OZMO

Thank you, David, Kyle, and Matthew

David Bates

Director of Innovation

Kyle Krcmaric

Software Engineer

Matthew Seiler

Senior Software Engineer

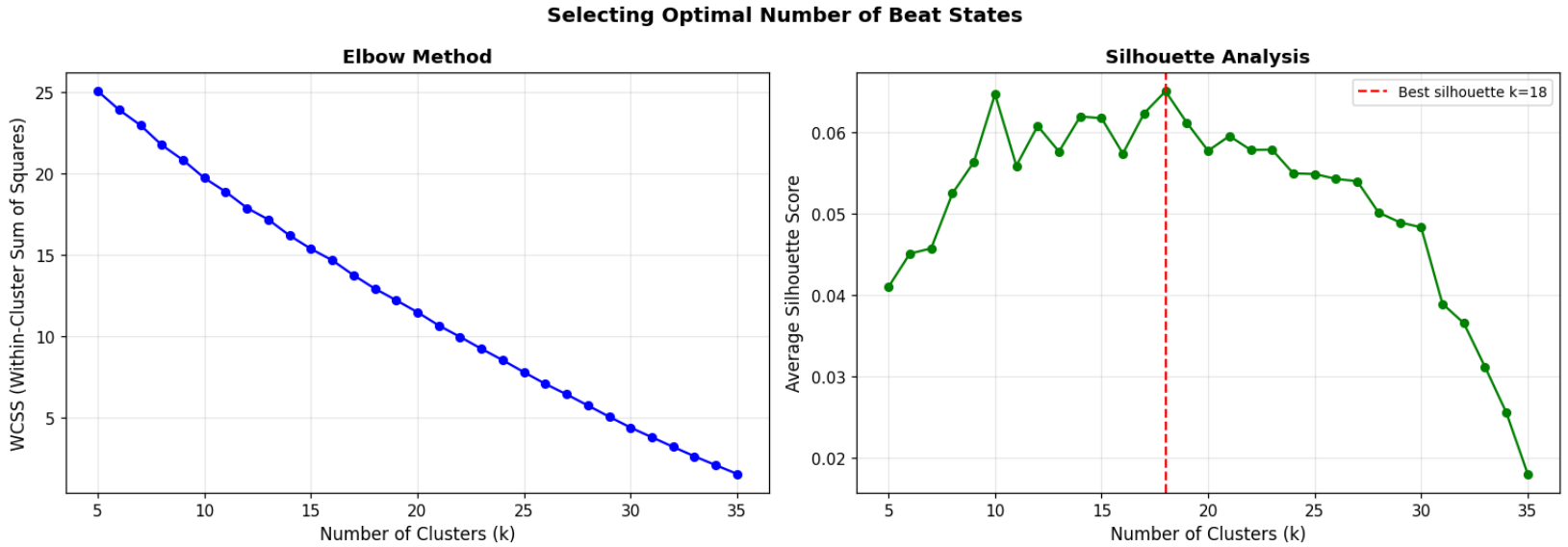
This work would not exist without your mentorship.

Appendix: Backup Slides for Q&A

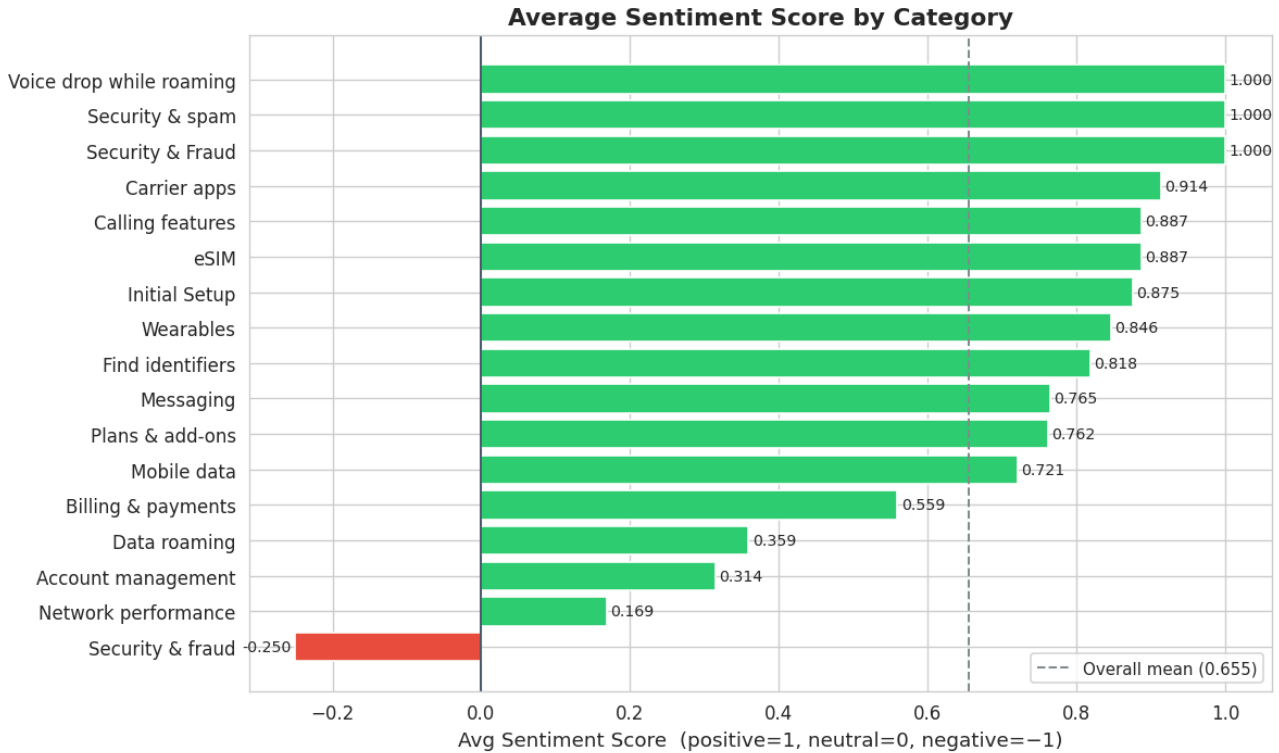
A1 · Full Databricks architecture

- INGEST → Delta Lake (telecom_data_raw, 19,322 rows)
- TRANSFORM → PII masking via ai_mask() · sub-sampled to 1,000 calls
- SPLIT → 895 train / 100 holdout (stratified by category)
- BEATS → TF-IDF + KMeans (k=18) · LLM-labelled clusters
- WORKFLOWS → QA-CoT structured synthesis (Llama 3.1 8B)
- RETRIEVE → Mosaic Vector Search (databricks-gte-large-en)
- SERVE → Call_Dashboard notebook · agent UI
- EVALUATE → 100-call holdout · per-call F1, Recall, Precision

A2 · Cluster-count selection (elbow + silhouette)



A3 · Average sentiment score by category



A4 · Beat zone analysis (3-way classifier)

Two signals on resolved calls: resolution rate AND beat consistency

Zone	Res. rate	Consistency	What it means for the dev team
HOT ZONE	≥ mean	≥ mean	Effective AND repeatable · promote the playbook
UNCHARTED	≥ mean	< mean	Resolves, no shared method · discover the playbook
DEAD AREA	< mean	(any)	Playbook is broken · redesign before retrieval

OZMO

A5 · QA-CoT workflow example: eSIM

- Trigger phrases: 'set up eSIM', 'transfer my line to a new device'

- Step 1 Verify account holder identity
- Step 2 Confirm last 4 SSN digits
- Step 3 Check eSIM compatibility for device model
- Step 4 Access QR code in Ozmo app
- Step 5 Scan QR + add cellular plan
- Step 6 Confirm caller ID + primary line selection
- Step 7 Test cellular data + voice

- Escalation if: device unsupported · QR fails · dual-eSIM conflict

OZMO

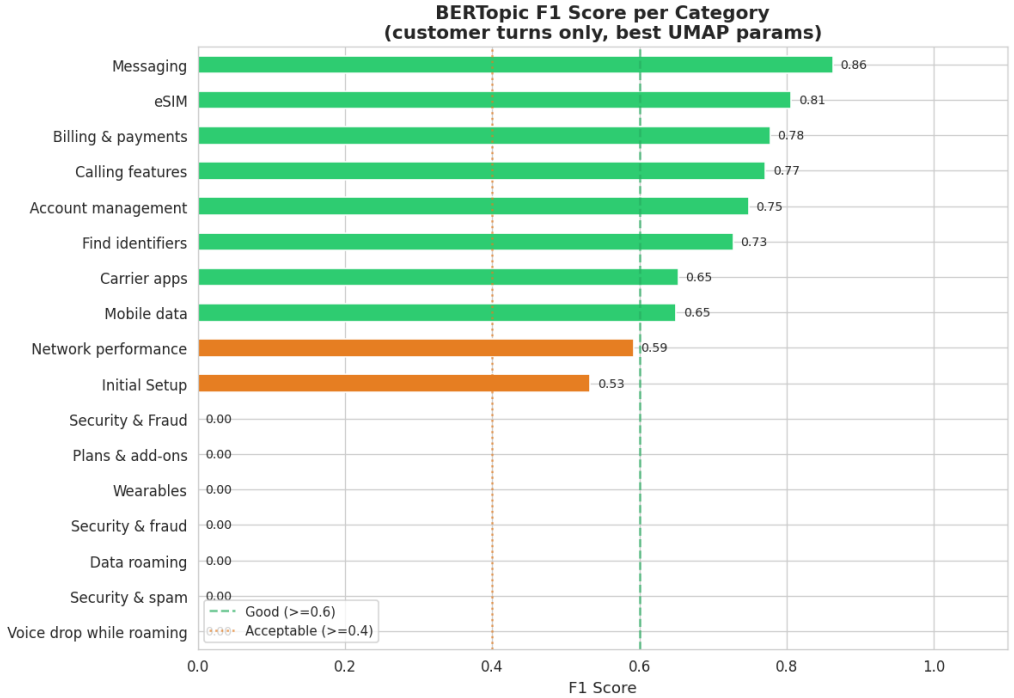
A6 · Holdout evaluation: per-category F1

- 100 held-out calls, transformed with FROZEN TF-IDF + KMeans models.
- RAG queried with category as filter; F1 = beat overlap with retrieved playbook.

- F1 highest in: Find Identifiers · Carrier Apps · eSIM
- F1 lowest in: Security & Fraud · Billing & Payments · Account Mgmt

- Expected pattern: where the playbook exists, retrieval works.

BERTopic for unsupervised category discovery — skip it



Rare classes · Label collisions · Vocabulary overlap → 7 of 15 categories at F1 = 0

A7 · PII masking: 8 entity types

- ai_mask() entity types:
- person · phone_number · email · ssn · credit_card_number
- date_of_birth · address · account_number

- Mask runs in SQL, server-side. No raw PII reaches an LLM.
- Masking failure → BLOCK suggestion · log · alert.

A8 · Cost & latency profile (Databricks endpoints)

- Embedding: one-time index build + delta refresh
- Vector Search: sub-second p50; p95 dependent on endpoint tier
- LLM: ~700 tokens × 2 calls per simulated call

- Graceful degradation:
- LLM down → fall back to canonical playbook (static Delta)
- Index sync fail → last-good index continues serving
- Empty retrieval → progressive filter fallback

OZMO

A9 · Cluster-count sensitivity

- k = 12 → beats too coarse; merges 'verify identity' with 'pull billing'
- k = 18 → selected (silhouette peak, elbow agrees)
- k = 24 → beats over-fragment; splits 'enable wifi calling' into 3 sub-beats

- Pipeline outputs are stable across k=15 to k=21.

A10 · BERTopic confusion matrix: what collided with what



A11 · Agent adoption plan

Stage	Who	What they do
Shadow	All agents (passive)	Suggestions logged but never shown
Canary	1–2 senior agents, opt-in	Receive live suggestions · shift feedback
A/B test	Randomized cohort (~50/50)	Compare AHT · transfer · callback

Always-on safeguards:

Safeguard	Why it exists
Per-agent opt-out toggle	Agent can disable assist at any time
Real-time supervisor view	Floor lead sees 'system flagged' counter
Weekly override review	Drives the retraining backlog

A12 · Scaling profile: 1K to 1M calls

Dimension	1K (capstone)	100K (canary)	1M (production)
Delta storage	~50 MB	~5 GB	~50 GB
Embedding cost	one-time	per-day batch	per-hour batch
Vector search latency	<1s p50	<1s p50*	Mosaic SLA*
Bottleneck	none	embed throughput	index refresh
Mitigation	n/a	tiered endpoint	triggered sync

**** Projected, not measured. Production validation is the canary's job.***

OZMO

A13 · Failure modes & graceful degradation

Failure mode	What the system does
LLM endpoint slow / down	Fall back to canonical playbook
Vector index sync failure	Last-good index continues serving
Empty retrieval (strict filter)	Progressive fallback: strict → any → unfiltered
Out-of-vocabulary call	Flag for SME review · suggest nothing
PII mask failure	BLOCK suggestion · log · alert

The agent always sees SOMETHING useful, never garbage.

A14 · Portability beyond telecom

Reusable (the machinery)	Telecom-specific (the content)
ETL + PII masking pipeline	Beat taxonomy labels
Beat clustering methodology	Issue category list
Hot / Dead zone classifier	Sub-category dictionary
QA-CoT workflow synthesis	Trigger phrases
Mosaic RAG architecture	(re-train per vertical)
Shadow → Canary → A/B rollout	n/a

Re-running on a new vertical: ~1 week once data is masked.

Why beat clustering, and how we landed on k=18

Approach considered	Why we didn't pick it
LLM-only labelling per utterance	Cost scales linearly · drift between runs
Sentence-BERT clustering	Heavier, slower · marginal gains here
Supervised classification	No utterance-level labels exist
TF-IDF + KMeans (selected)	Cheap · deterministic · LLM labels 18 centroids

Pay the LLM cost ONCE per cluster (18 calls), not once per utterance (9,604).

k = 18 chosen by silhouette peak. Stable across k = 15–21.

OZMO

Why RAG instead of fine-tuning

Dimension	Fine-tuned LLM	RAG over playbooks (selected)
Cost	Train + host custom model	Use existing endpoint
Updates	Re-train cycle (days)	Index sync (minutes)
Governance	Hard to inspect	Every suggestion traces to a call
Explainability	Black-box	'This is from CALL-21838093'
Failure mode	Hallucination invisible	Empty retrieval → fallback

Governance and explainability tipped the choice. Not cost.

What didn't work: BERTopic for unsupervised category discovery

RARE CLASSES

too few docs to seed clusters

LABEL COLLISIONS

Fraud / spam / fraud collide

VOCABULARY OVERLAP

Plans absorbed by Data Roaming

OZMO

Stage Group 1: Ingest & Govern



Extraction

Raw transcript JSON → Delta Lake (one row per call)

Transformation

Flatten turns · ai_mask() across 8 PII types

WHY THIS GROUP EXISTS

- Data hygiene happens at the BOUNDARY, never deep in the pipeline.
- Holdout-first prevents test-set contamination of the RAG index.
- PII never travels: redacted at the SQL layer, not at the LLM call.

Stage Group 2: Understand

INGEST & GOVERN → **UNDERSTAND** → SYNTHESIZE → SERVE

Sentiment Analysis

ai_analyze_sentiment() per call · tone signal

Beat Extraction

Cluster every utterance into 18 reusable actions

Beat Zone Analysis

Hot Zone / Uncharted / Dead Area classifier

WHY THIS GROUP EXISTS

- Prose is impossible to query. Structured signals are easy to query.
- Beats are reusable across calls; sentences aren't.
- Zone classification asks 'does a playbook exist?', sharper than 'is this category good?'

Stage Group 3: Synthesize



LLM Beat Analysis

8-question Guide/Implementer with branching workflow

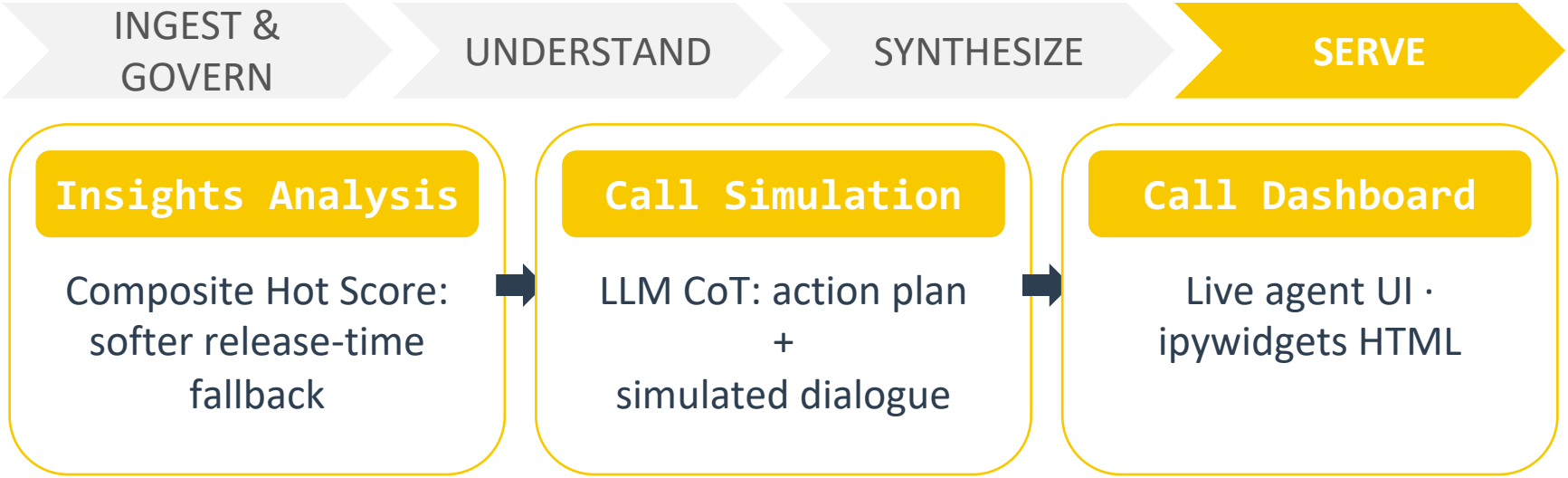
Call Retrieval

Vector index of all calls · filterable · sub-second

WHY THIS GROUP EXISTS

- Bottom-up beats give us reality. Top-down QA-CoT gives us a usable workflow.
- RAG over fine-tuning: every suggestion traces to a specific resolved call.
- Mosaic Delta Sync = index updates as the data changes. No bespoke infra.

Stage Group 4: Serve



WHY THIS GROUP EXISTS

- A method nobody uses isn't a method. The dashboard is the proof of usability.
- Three suggestions, agent picks one, or none. Decision authority stays human.

Synthetic data, AI-generated, with a known positivity bias

1,000
calls

15
categories

38
agents

sub-sampled from 19,322 raw records · AI-generated synthetic transcripts (methodology validation only)

Sentiment skews positive: 523 of 1,000 are Positive; only 49 are Negative.

Recalibrate against production transcripts before trusting absolute numbers.

The methodology is the deliverable. The numbers need a real-data re-run.

OZMO

Guide Questions (1 of 2)

1. **Entry Conditions**
   What customer-reported symptoms or complaints indicate this issue type? What would a customer say to trigger this workflow?
2. **Initial Verification**
   What information does the agent collect or verify at the start of the call, before attempting any fix? List each verification step in order.
3. **Primary Resolution**
   What is the most common sequence of steps agents take to resolve this issue? Describe each step as a discrete action the agent performs.
4. **Branch Conditions**
   Where does the agent's next action depend on customer input or a system lookup? For each branch point, what are the conditions and what does the agent do differently?

Guide Questions (2 of 2)

5. **Failure Paths**
   Based on transferred and callback calls, where does resolution break down? What does the agent attempt that fails, and what happens next?
6. **Escalation Triggers**
   What conditions — stated by the customer or found during troubleshooting — cause the agent to transfer or escalate? What information should be passed to the next tier?
7. **Success Criteria**
   How does the agent confirm the issue is resolved before closing the call? What does the customer say or do that confirms success?
8. **Edge Cases**
   Are there unusual customer situations, device types, account states, or environmental factors that require a significantly different approach from the primary path?